

東京 2025 LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

Rahul Rameshbabu

Tackling challenges with HID and related device driver support in Rust



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Quick Background

HID patches posted upstream

- Patches were posted a while back for HID driver bindings on the Rust for Linux mailing list.
  - https://lore.kernel.org/rust-for-linux/20250313160220.6410-2-sergeantsagara@protonmail.com/
- The initial intent of these patches was to support probing complex HID devices that would require resource management and interfacing with other kernel subsystems.
- In later iterations, these patches were simplified to only do HID report fixup for a Glorious Model O- mouse.



Quick Background

Work being done out of tree

- Expanding on HID Rust driver development in the repository below
  - https://github.com/Binary-Eater/linux
- Have been more focused on reverse engineering hardware that would be supported by a Rust-written driver
  - Looking at the HID controller on RazerBlades, used for various device controls such as the keyboard backlight.
    - Using USBPcap to trace what is happening when Razer Synapse configures the device.
  - There has not been further Rust binding code since my latest RFCs due to this reverse engineering effort
- Maybe it would be better to start writing stub code to post as RFCs while in parallel to reverse engineering?



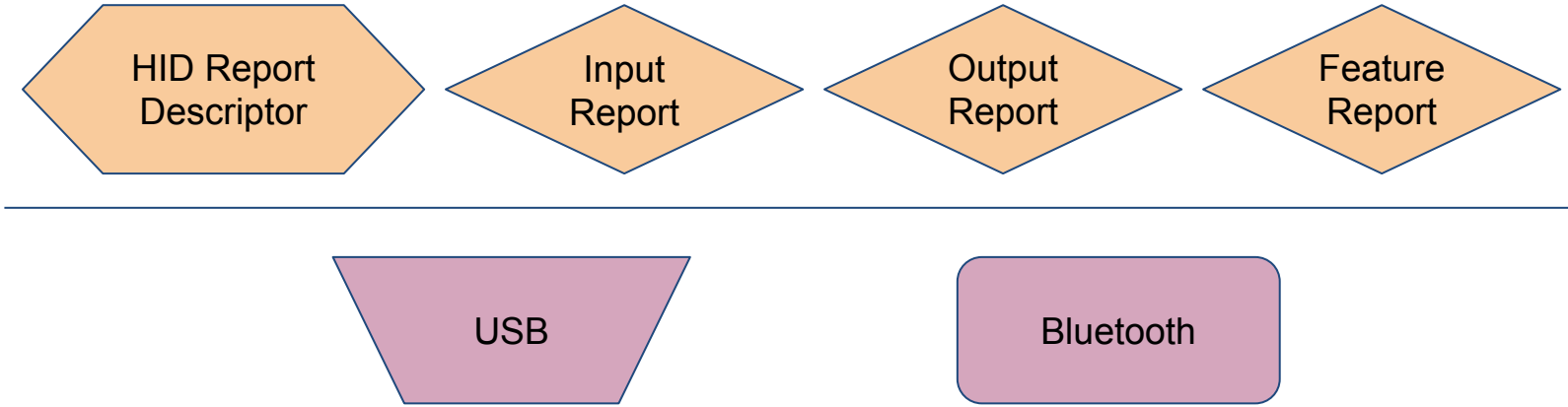
Quick Background

Original Intent

- Planned on doing DRM monitor control related properties in Rust.
    - Concern about ecosystem maturity for doing core kernel features in Rust came up
        - One example was the current state of gccrs
        - More details here: https://binary-eater.github.io/kangrejos-2025/

What is HID?

A higher level abstraction for communication with hardware involved with human interactions



Refer to https://www.usb.org/hid for more details.







What is HID?

Example of a HID report descriptor

```
Usage Page (Generic Desktop),           ;Use the Generic Desktop Usage Page
Usage (Mouse),
    Collection (Application),           ;Start Mouse collection
    Usage (Pointer),
    Collection (Physical),              ;Start Pointer collection
      Usage Page (Buttons)
      Usage Minimum (1),
      Usage Maximum (3),
      Logical Minimum (0),
      Logical Maximum (1),              ;Fields return values from 0 to 1
      Report Count (3),
      Report Size (1),                  ;Create three 1 bit fields (button 1, 2, & 3)
      Input (Data, Variable, Absolute), ;Add fields to the input report.
      Report Count (1),
      Report Size (5),                  ;Create 5 bit constant field
      Input (Constant),                 ;Add field to the input report
      Usage Page (Generic Desktop),
      Usage (X),
      Usage (Y),
      Logical Minimum (-127),
      Logical Maximum (127),            ;Fields return values from -127 to 127
      Report Size (8),
      Report Count (2),                 ;Create two 8 bit fields (X & Y position)
      Input (Data, Variable, Relative), ;Add fields to the input report
    End Collection,                     ;Close Pointer collection
End Collection                          ;Close Mouse collection
```

Coding Challenges Faced In the Upstream Patches

Converting between kernel::hid::Group and the HID_GROUP_* define equivalents

```
/*
 * HID device groups
 *
 * Note: HID_GROUP_ANY is declared in linux/mod_devicetable.h
 * and has a value of 0x0000
 */
#define HID_GROUP_GENERIC               0x0001
#define HID_GROUP_MULTITOUCH            0x0002
#define HID_GROUP_SENSOR_HUB            0x0003
#define HID_GROUP_MULTITOUCH_WIN_8      0x0004

/*
 * Vendor specific HID device groups
 */
#define HID_GROUP_RMI                       0x0100
#define HID_GROUP_WACOM                     0x0101
#define HID_GROUP_LOGITECH_DJ_DEVICE        0x0102
#define HID_GROUP_STEAM                     0x0103
#define HID_GROUP_LOGITECH_27MHZ_DEVICE     0x0104
#define HID_GROUP_VIVALDI                   0x0105
```

include/linux/hid.h

The HID device groups are used to categorize the numerous input devices out there.

The main value of doing this is to detect some information common to a group of devices and automatically apply a set of quirks or HID report handling logic common to the group.

Let's say multitouch devices require a certain HID report fixup to behave properly. We can detect a HID device is multitouch by scanning for its feature HID usage to see if multitouch is set and then mapping the probed device to the relevant HID group.

Without the multitouch HID group, any new trackpads created would need to manually have a multitouch quirk applied. Imagine having to wait for a kernel dev. to patch your trackpad and be required to upgrade kernels just to get multitouch functionality on new trackpads.

Converting between kernel::hid::Group and the HID_GROUP_* define equivalents

```rust
/// HID device groups are intended to help categories HID devices based on a set
/// of common quirks and logic that they will require to function correctly.
#[repr(u16)]
pub enum Group {
    /// Used to match a device against any group when probing.
    Any = bindings::HID_GROUP_ANY as u16,

    /// Indicates a generic device that should need no custom logic from the
    /// core HID stack.
    Generic = bindings::HID_GROUP_GENERIC as u16,

    /// Maps multitouch devices to hid-multitouch instead of hid-generic.
    Multitouch = bindings::HID_GROUP_MULTITOUCH as u16,

    /// Used for autodetecing and mapping of HID sensor hubs to
    /// hid-sensor-hub.
    SensorHub = bindings::HID_GROUP_SENSOR_HUB as u16,

    /// Used for autodetecing and mapping Win 8 multitouch devices to set the
    /// needed quirks.
    MultitouchWin8 = binding::HID_GROUP_MULTITOUCH_WIN_8 as u16,
```

rust/kernel/hid.rs (not merged in tree)

To have the equivalent of the C HID groups in a Rust type that best represents the information, manual conversion of the generated Rust bindings to a Rust enum is required.

The issue is that the C HID group defines get represented as u32 in Rust when using bindgen. However, the binded C apis in Rust will expect a u16 for the HID groups. Rust's "as" keyword is used to trivially cast the u32 values as u16 since the values do not exceed the u16 max value.

The bindgen typing problem extends to HID report items

Similar to the previous example, HID items are defines in C. bindgen maps them to u32 in Rust. However, the APIs to work with them expect u8.

```rust
/// Indicates the item is static read-only.
///
/// Refer to [Device Class Definition for HID 1.11]
/// Section 6.2.2.5 Input, Output, and Feature Items.
///
/// [Device Class Definition for HID 1.11]: https://www.usb.org/sites/default/files/hid1_11.pdf
pub const MAIN_ITEM_CONSTANT: u8 = bindings::HID_MAIN_ITEM_CONSTANT as u8;
```

On the other hand, having to explicitly map these constants creates an opportunity to write more verbose documentation in Rust doc comments.

Should bindgen map C #define integer constant expressions (ICE) to the smallest integer type that can express the value?

Using kernel::hid::Group in C binded APIs and structures

Creating a new USB HID device id

To declare a USB HID device that a driver written in Rust would probe, we implement the new_usb function.

This function takes a kernel::hid::Group, a u32 vendor, and a u32 product. It then constructs a kernel::hid::DeviceId. This structure is just a transparent representation of the C struct hid_device_id.

struct hid_device_id consumes a u16 group rather than kernel::hid::Group. Naturally, we would just implement the From trait to convert kernel::hid::Group to a u16, right?

```rust
/// Abstraction for the HID device ID structure `struct hid_device_id`.
#[repr(transparent)]
#[derive(Clone, Copy)]
pub struct DeviceId(bindings::hid_device_id);

impl DeviceId {
    /// Equivalent to C's `HID_USB_DEVICE` macro.
    ///
    /// Create a new `hid::DeviceId` from a group, vendor ID, and device ID
    /// number.
    pub const fn new_usb(group: Group, vendor: u32, product: u32) -> Self {
        Self(bindings::hid_device_id {
            bus: 0x3, // BUS_USB
            group: group.into(),
            vendor,
            product,
            driver_data: 0,
        })
    }
```

We cannot implement the From trait for u16 to convert from kernel::hid::Group

new_usb is a const fn

Because new_usb is a const fn, only other const functions can be called within it. new_usb has to be a const fn. Otherwise, it cannot be used to populate a compile-time defined DeviceId table representing the devices that a driver probes.

The From trait does not implement a const conversion function. This makes the From trait not viable for use as part of DeviceId table construction. However, the proposed const_trait_impl feature would resolve this problem.

https://github.com/rust-lang/rust/issues/143874

```
// TODO: use `const_trait_impl` once stabilized:
//
// ```
// impl const From<Group> for u16 {
//     /// [`Group`] variants are represented by [`u16`] values.
//     fn from(value: Group) -> Self {
//         value as Self
//     }
// }
// ```
impl Group {
    /// Internal function used to convert [`Group`] variants into [`u16`].
    const fn into(self) -> u16 {
        self as u16
    }
}
```

Getting a kernel::hid::Group from C interfaces

```rust
/// The HID device representation.
///
/// This structure represents the Rust abstraction for a C `struct hid_device`.
/// The implementation abstracts the usage of an already existing C `struct
/// hid_device` within Rust code that we get passed from the C side.
///
/// # Invariants
///
/// A [`Device`] instance represents a valid `struct hid_device` created by the
/// C portion of the kernel.
#[repr(transparent)]
pub struct Device<Ctx: device::DeviceContext = device::Normal>(
    Opaque<bindings::hid_device>,
    PhantomData<Ctx>,
);

impl<Ctx: device::DeviceContext> Device<Ctx> {
    /// Returns the HID report group.
    pub fn group(&self) -> Group {
        // SAFETY: `self.as_raw` is a valid pointer to a `struct hid_device`
        unsafe { *self.as_raw() }.group.into()
    }
```

Using the From trait to convert from a u16

Unlike with the case of new_usb, there are no contexts currently where going from a u16 to a kernel::hid::Group is const.

We can cleanly use the From trait to turn the u16 from the struct hid_device into a kernel::hid::Group.

Everything looks good, right?

Getting a kernel::hid::Group from C interfaces

```rust
impl From<u16> for Group {
    /// [`u16`] values can be safely converted to [`Group`] variants.
    fn from(value: u16) -> Self {
        match value.into() {
            bindings::HID_GROUP_GENERIC => Group::Generic,
            bindings::HID_GROUP_MULTITOUCH => Group::Multitouch,
            bindings::HID_GROUP_SENSOR_HUB => Group::SensorHub,
            bindings::HID_GROUP_MULTITOUCH_WIN_8 => Group::MultitouchWin8,
            bindings::HID_GROUP_RMI => Group::RMI,
            bindings::HID_GROUP_WACOM => Group::Wacom,
            bindings::HID_GROUP_LOGITECH_DJ_DEVICE => Group::LogitechDJDevice,
            bindings::HID_GROUP_STEAM => Group::Steam,
            bindings::HID_GROUP_LOGITECH_27MHZ_DEVICE => Group::Logitech27MHzDevice,
            bindings::HID_GROUP_VIVALDI => Group::Vivaldi,
            _ => Group::Any,
        }
    }
}
```

Conversion requires a one-to-one pattern match

First, we take the u16 value from the C interface and use the u32 From trait to convert the u16 to a u32. This is required since bindgen treats C #define integer constant expressions as u32.

Next, we match the C #define value to the equivalent Group value. The actual values are identical but for the sake of type strictness we perform the pattern match.

We also introduce a bug in this pattern match that is tough to catch. If the C API defines any new groups, we may incorrectly map those new groups to Group::Any.

Can we do better than "_ => Group::Any"?

```
match value.into() {
    bindings::HID_GROUP_ANY => Group::Any,
    bindings::HID_GROUP_GENERIC => Group::Generic,
    bindings::HID_GROUP_MULTITOUCH => Group::Multitouch,
    bindings::HID_GROUP_SENSOR_HUB => Group::SensorHub,
    bindings::HID_GROUP_MULTITOUCH_WIN_8 => Group::MultitouchWin8,
    bindings::HID_GROUP_RMI => Group::RMI,
    bindings::HID_GROUP_WACOM => Group::Wacom,
    bindings::HID_GROUP_LOGITECH_DJ_DEVICE => Group::LogitechDJDevice,
    bindings::HID_GROUP_STEAM => Group::Steam,
    bindings::HID_GROUP_LOGITECH_27MHZ_DEVICE => Group::Logitech27MHzDevice,
    bindings::HID_GROUP_VIVALDI => Group::Vivaldi,
    _ => panic!("Unknown HID group {} found", value),
}
```

Now we panic the kernel

If an unknown HID group is encountered, we now will crash the running kernel. Quite the improvement..... This option is not viable.

Can we do better than "_ => Group::Any"?

```
impl const From<u16> for Group {
    /// [`u16`] values can be safely converted to [`Group`] variants.
    fn from(value: u16) -> Self {
        match value.into() {
            bindings::HID_GROUP_ANY => Group::Any,
            bindings::HID_GROUP_GENERIC => Group::Generic,
            bindings::HID_GROUP_MULTITOUCH => Group::Multitouch,
            bindings::HID_GROUP_SENSOR_HUB => Group::SensorHub,
            bindings::HID_GROUP_MULTITOUCH_WIN_8 => Group::MultitouchWin8,
            bindings::HID_GROUP_RMI => Group::RMI,
            bindings::HID_GROUP_WACOM => Group::Wacom,
            bindings::HID_GROUP_LOGITECH_DJ_DEVICE => Group::LogitechDJDevice,
            bindings::HID_GROUP_STEAM => Group::Steam,
            bindings::HID_GROUP_LOGITECH_27MHZ_DEVICE => Group::Logitech27MHzDevice,
            bindings::HID_GROUP_VIVALDI => Group::Vivaldi,
            _ => build_error!("Unknown HID group encountered"),
        }
    }
}
```

Is this the perfect solution?

This solution makes use of the 'const_trait_impl' proposed feature in Rust. We then call on build_error! to throw an error if an unknown HID group is encountered. This seems perfect!

Well actually, this solution is just as problematic if not more problematic than the previous solution shown. First of all, there is no context where conversion from a u16 to a kernel::hid::Group is done at compile time. What does that mean for build_error! ? It's just a panic! call, except this implementation is trickier to realize this since panic! is not directly called.

Refer to rust/kernel/build_assert.rs

Can we do better than "_ => Group::Any"?

```rust
impl TryFrom<u16> for Group {
    type Error = &'static str;

    /// [`u16`] values can be safely converted to [`Group`] variants.
    fn from(value: u16) -> Result<Self, Self::Error> {
        match value.into() {
            bindings::HID_GROUP_ANY => Ok(Group::Any,
            bindings::HID_GROUP_GENERIC => Ok(Group::Generic),
            bindings::HID_GROUP_MULTITOUCH => Ok(Group::Multitouch),
            bindings::HID_GROUP_SENSOR_HUB => Ok(Group::SensorHub),
            bindings::HID_GROUP_MULTITOUCH_WIN_8 => Ok(Group::MultitouchWin8),
            bindings::HID_GROUP_RMI => Ok(Group::RMI),
            bindings::HID_GROUP_WACOM => Ok(Group::Wacom),
            bindings::HID_GROUP_LOGITECH_DJ_DEVICE => Ok(Group::LogitechDJDevice),
            bindings::HID_GROUP_STEAM => Ok(Group::Steam),
            bindings::HID_GROUP_LOGITECH_27MHZ_DEVICE => Ok(Group::Logitech27MHzDevice),
            bindings::HID_GROUP_VIVALDI => Ok(Group::Vivaldi),
            _ => Err("Unknown HID group encountered"),
        }
    }
}
```

Can we use the TryFrom trait instead of From?

As an improvement to previous iterations, this solution will not panic the kernel. However, this solution negatively impacts the caller experience. In C, a caller can trivially just check the group field of struct hid_device or struct hid_device_id.

This solution forces Rust drivers to handle this error case just because there is no good way to directly map the C defines to a Rust type.

The need for “getters” from C structs such as “struct hid_device”

Can these somehow be autogenerated?

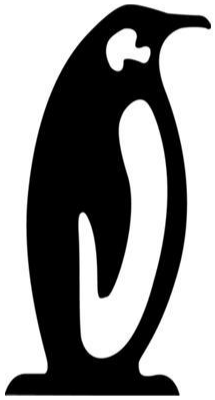
On one hand, the getters for bus, vendor, and product are repetitive. On the other side of this, the getter for group is different because it needs to call the into() function generated by the From trait.

Maybe the best balance would be to have a declarative macro that simplifies the creation of these getters?

```rust
impl<Ctx: device::DeviceContext> Device<Ctx> {
    /// Returns the HID transport bus ID.
    pub fn bus(&self) -> u16 {
        // SAFETY: `self.as_raw` is a valid pointer to a `struct hid_device`
        unsafe { *self.as_raw() }.bus
    }

    /// Returns the HID vendor ID.
    pub fn vendor(&self) -> u32 {
        // SAFETY: `self.as_raw` is a valid pointer to a `struct hid_device`
        unsafe { *self.as_raw() }.vendor
    }

    /// Returns the HID product ID.
    pub fn product(&self) -> u32 {
        // SAFETY: `self.as_raw` is a valid pointer to a `struct hid_device`
        unsafe { *self.as_raw() }.product
    }
```



東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

